

网络层（控制平面）

1.路由选择算法

1.1 路由概念

路由：按照某种指标找到一条源节点到目标节点的较好的路径

指标：跳数、延迟、费用、队列长度等

- 以网络为单位进行路由，路由信息传输、计算和匹配的代价低；路由就是计算网络到其他网络如何走
- 网络到网络的路由其实就是路由器到路由器的路由
- 路由选择算法：网络层软件的一部分，完成路由
- 网络可以抽象成图，路由器表示节点，链路代价为边；路由的输入为拓扑、边的代价、源节点，输出为源节点的最小生成树

1.2 路由原则

- 正确性
- 简单性
- 健壮性：能适应通信量和网络拓扑的变化
- 稳定性：产生的路由不应该摇摆
- 公平性：对每一个站点都公平
- 最优性

1.3 路由算法分类

1. 全局的：所有的路由器拥有完整的拓扑和边的代价的信息：link state 算法
2. 分布式：路由器只知道与它有物理连接关系的邻居路由器，和到相应邻居路由器的代价值；叠代地与邻居交换路由信息：distance vector 算法
3. 静态：路由随时间变换缓慢：非自适应算法
4. 动态：路由变化快：自适应算法

1.4 LS

LS的工作过程:

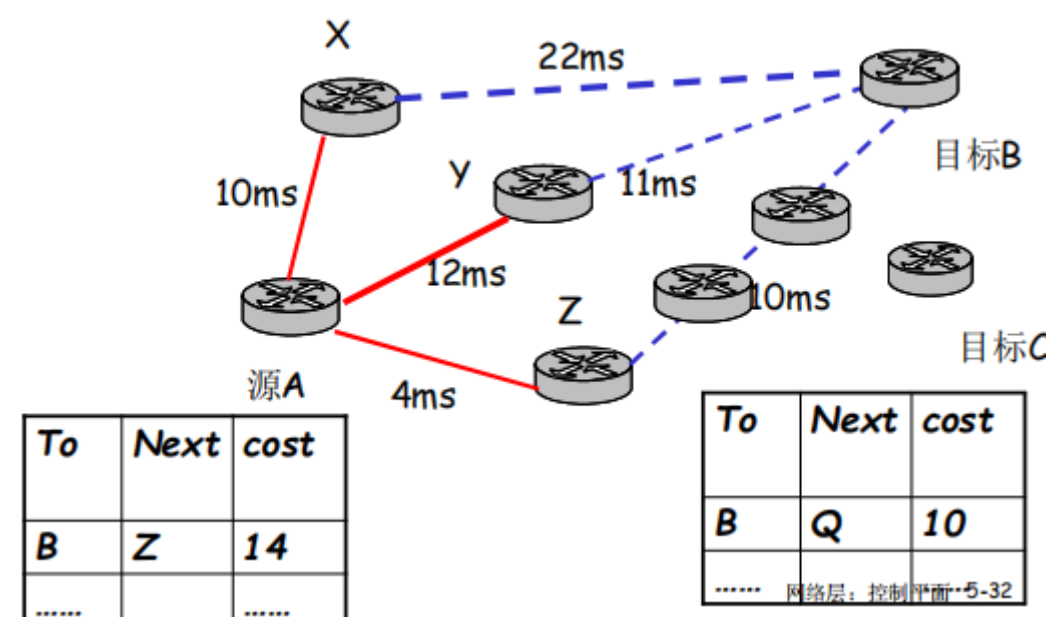
□ LS路由的基本工作过程

1. 发现相邻节点,获知对方网络地址
2. 测量到相邻节点的代价(延迟,开销)
3. 组装一个LS分组,描述它到相邻节点的代价情况
4. 将分组通过扩散的方法发到所有其它路由器
以上4步让**每个**路由器获得**拓扑**和**边代价**
5. 通过Dijkstra算法找出最短路径（这才是路由算法）
 1. 每个节点独立算出来到其他节点（路由器=网络）的最短路径
 2. 迭代算法：第k步能够知道本节点到k个其他节点的最短路径

LS的问题:可能的震荡;代价变化整个网络拓扑需要不断计算新的路由

1.5 DV

基本思想:各路由器维护一张路由表;各路由器与相邻路由器交换路由表;根据获得的路由信息,更新路由表



Bellman-Ford方程(dp):

$$d_x(y) = \min\{c(x, v) + d_v(y)\}$$

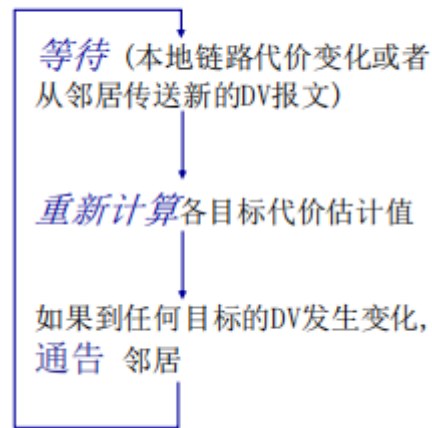
异步式, 迭代: 每次本地迭代被以下事件触发:

- 本地链路代价变化了
- 从邻居来了DV的更新消息

分布式:

- 每个节点只是在自己的DV改变之后向邻居通告
 - 然后邻居们在有必要的时候通知他们的邻居

每个节点:



- DV的无穷计算问题
- DV的水平分裂算法

1.6 LS和DV的比较

LS 和 DV 算法的比较

消息复杂度 (DV胜出)

- **LS:** 有 n 节点, E 条链路, 发送报文 $O(nE)$ 个
 - 局部的路由信息; 全局传播
- **DV:** 只和邻居交换信息
 - 全局的路由信息, 局部传播

健壮性: 路由器故障会发生什么 (LS胜出)

LS:

- 节点会通告不正确的链路代价
- 每个节点只计算自己的路由表
- 错误信息影响较小, 局部, 路由较健壮

收敛时间 (LS胜出)

- **LS:** $O(n^2)$ 算法
 - 有可能震荡
- **DV:** 收敛较慢
 - 可能存在路由环路
 - **count-to-infinity** 问题

DV:

- DV 节点可能通告对全网所有节点的不正确路径代价
 - 距离矢量
- 每一个节点的路由表可能被其它节点使用
 - 错误可以扩散到全网

2种路由选择算法都有其优缺点, 而且在互联网上都有应用

2.因特网中自治系统(AS)内部的路由选择(IGP)

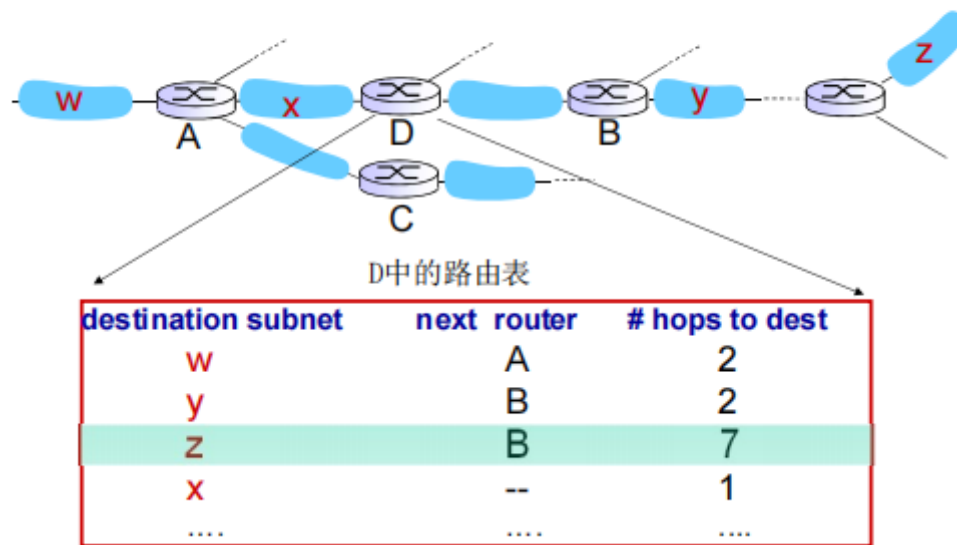
1.RIP (Routing Information Protocol)

在1982年的BSD-UNIX中实现。

使用DV算法:

- 每条链路代价为1, 最大跳数为15hops
- DV每隔30秒和邻居交换DV, 通告
- 每个通告包括最多25个目标子网

RIP: 例子



链路失效和恢复:

- 如果180秒没有收到通告信息: 邻居或者链路失效
- 好消息传得快, 坏消息传得慢
- 使用毒性逆转组织ping-pong回路 (不可达的距离: 16hops)

关于毒性逆转参考:[毒性逆转](#)

RIP进程处理:

- RIP以应用进程的方式实现: `routed(daemon)`

`routed`是一个守护进程, **用于管理和维护系统的路由表**。它负责监视网络接口的状态变化, 并更新路由表以确保网络通信的顺利进行。`routed`还可以处理路由表的添加、删除和修改操作, 以及处理路由表的更新和同步。它是Linux系统中的一个重要组件, 用于确保网络通信的高效性和稳定性。

- 通告报文通过UDP报文传到达,周期性重复
- 网络层的协议使用了传输层的服务,以应用层实体的方式实现

2.OSPF(Open Shortest Path First)

使用LS算法:

- LS分组在网络中(AS内部)分发
- 全局网路拓扑,代价在每一个节点中都保持
- 路由计算使用Dijkstra算法

OSPF通告信息中携带每一个邻居路由器一个表项

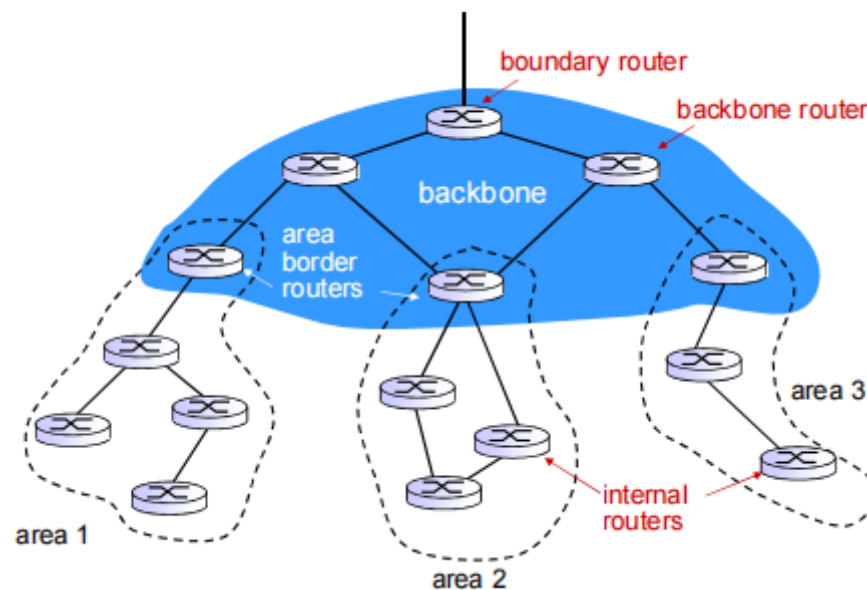
通告信息会传遍AS内部(通过泛洪):在IP数据包报上直接传送OSPF报文,而不是通过UDP或TCP

OSPF的高级特性:

- 安全:所有的OSPF报文都是经过认证的
- 允许有多个代价相同的路径存在
- 对于每一个链路,对于不同的TOS(type of service)有多重代价矩阵

- 对单播和多播的集成支持
- 在大型网络中支持层次性OSPF

层次化的OSPF:



- 2个级别的层次:本地,骨干
链路状态通告仅仅在本地范围内进行
每一个节点拥有本地区域的拓扑信息(对于其他区域,知道去它的方向,通过区域边界路由器)
- 区域边界路由器:汇总(gather)到自己区域内网络的距离,向其他区域边界路由器通告
- 骨干路由器:仅仅在骨干网络中,运行OSPF路由
- 边界路由器:连接其他的AS

3.ISP之间的路由选择: BGP

3.1 层次路由

平面路由的问题: 网络规模太大, 路由信息的存储、传输和计算代价巨大; 管理方面: 不同的网络所有者希望按照自己的方式管理网络, 并且对外隐藏网络细节, 同时还希望与其他网络互联。

层次路由: 将网络分成一个个的AS, 一个AS用一个AS Number标识, 一个ISP可能包括一个或多个AS。

- 两个层次: AS内部路由 (intra-AS) 不同的AS运行不同的内部网关协议; AS间运行AS间路由协议 (inter-AS):AS间的路由问题。

层次路由优点:

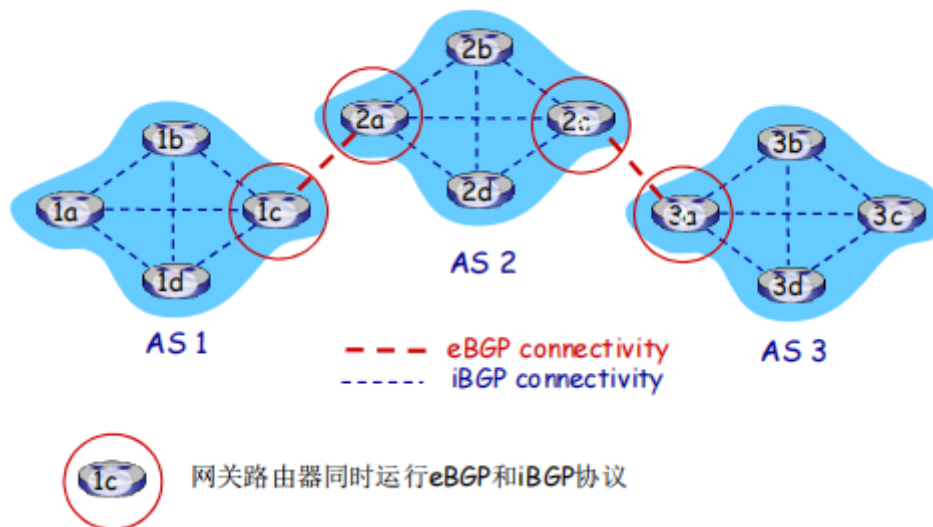
- 解决规模问题: AS内部规模可控, 路由器数量足够; AS之间扩展性强: 规模增大性能减少不多
- 解决管理问题: 各个AS可以运行不同的内部网关协议; 可以使自己网络的细节不向外透露

3.2 BGP (Border Gateway Protocol)

eBGP:从相邻的AS获得子网可达信息

iBGP:将获得的子网可达信息传遍AS内部的路由器

基于DV算法: 还包括到达各个目标网络的**详细路径 (AS序号的列表)** 能够避免DV算法的路由环路问题。

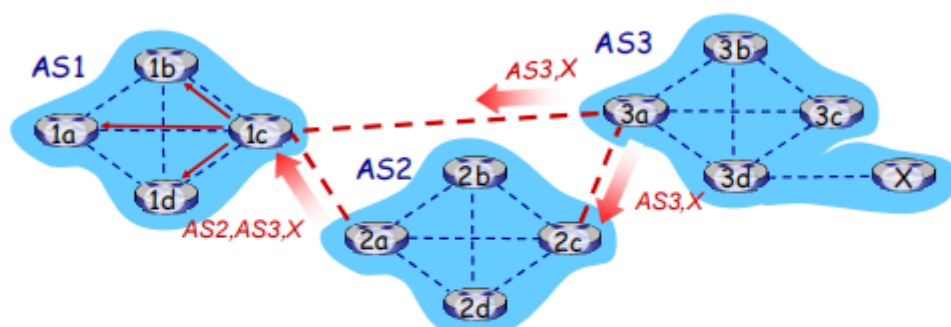


BGP会话: 2个BGP路由器在一个半永久的TCP连接上交换BGP报文，通告向不同子网前缀的路径（AS序列路径）。

路径的属性: AS路径列表，下一跳的指示。

BGP路径通告:

BGP 路径通告



网关路由器可能获取有关一个子网X的多条路径，从多个eBGP会话上：

- AS1 网关路由器1c从2a学习到路径: **AS2, AS3, X**
- AS1网关路由器1c从3a处学习到路径**AS3, X**
- 基于策略，AS1路由器1c选择了路径: **AS3, X**，而且通过iBGP告诉所有AS1内部的路由器

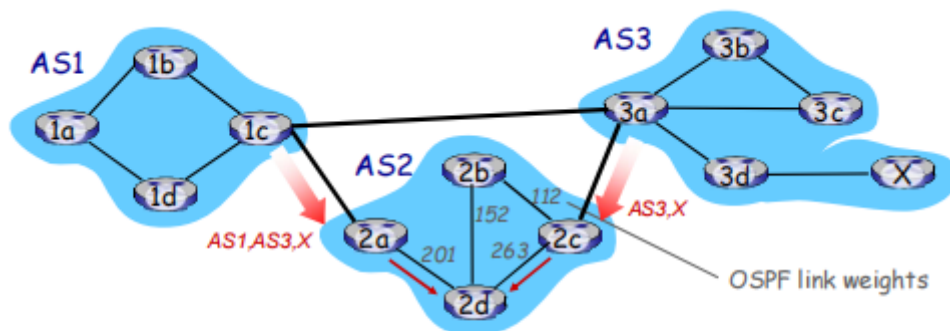
网络层：控制平面 5-78

BGP报文: OPEN、UPDATE、KEEPALIVE、NOTIFICATION

BGP路径选择:

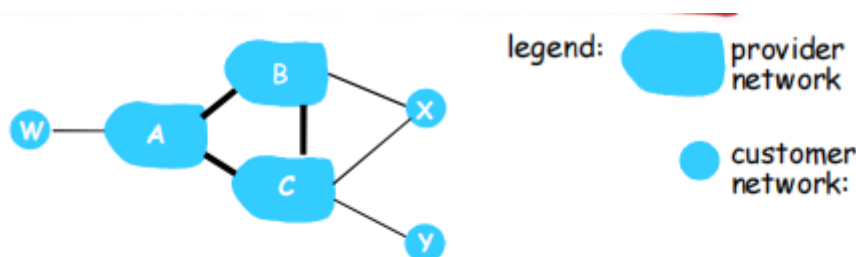
1. 本地偏好
2. 最短AS路径: AS的跳数
3. 最近的下一跳路由器: 热土豆路由
4. 附加的判据: 使用BGP标识

热土豆路由: 选择具备最小内部区域代价的网关作为出口；不考虑域间代价



- ❑ 2d通过iBGP获知，它可以通过2a或者2c到达X
- ❑ **热土豆策略**：选择具备最小内部区域代价的网关作为往X的出口（如：2d选择2a，即使往X可能有比较多的AS跳数）：不要操心域间的代价！

BGP：通告路径通告执行策略



假设一个ISP只想路由流量到/去往它的客户网络（不想承载其他ISPs之间的流量，即不通告：不是去往我的客户，也不是来自我的客户）

- A向B和C通告路径Aw
- B选择不向C通告BAw：
 - B从CBAw的路由上无法获得收益，因为C,A,w都不是B的客户
 - C从而无法获知CBAw路径的存在：每个ISP感知到的网络和真实不一致
- C可能会通过CAw（而不是使用B）最终路由到w

网络层：控制平面 5-84

4.SDN控制平面

关于SDN详见：[SDN](#)

SDN面临的挑战：

- 强化控制平面：可信、可靠、性能可扩展性、安全的分布式系统
- 网络、协议满足特殊任务的需求
- 互联网络范围内的扩展性：全网部署

5.ICMP

Internet Control Message Protocol:由主机、路由器、网关用于传达网络层控制信息。提供错误报告。echo请求和回复(ping)。

ICMP在IP数据报上承载。

关于ICMP详见：[ICMP](#)

6.网络管理和SNMP

6.1 网络管理

网络管理包括了硬件、软件和人类元素的设置，综合和协调，以便监测，测试，轮询，配置，分析，评价和控制网络和网元资源，用合理的成本满足实时性。

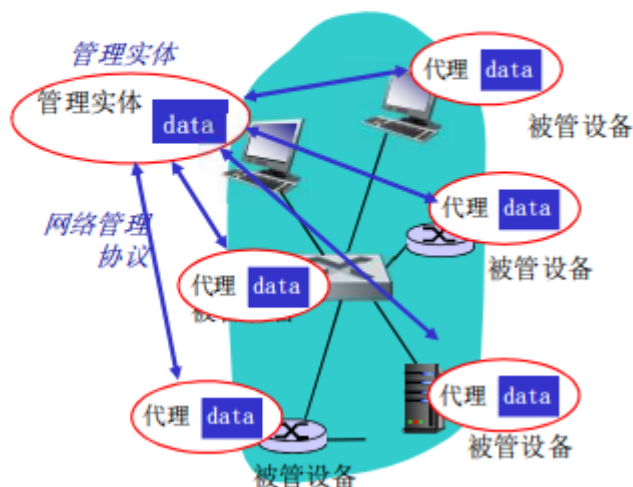
6.2 网络管理功能

- 性能管理
- 故障管理
- 配置管理
- 账户管理
- 安全管理

6.3 网络管理架构

网络管理架构

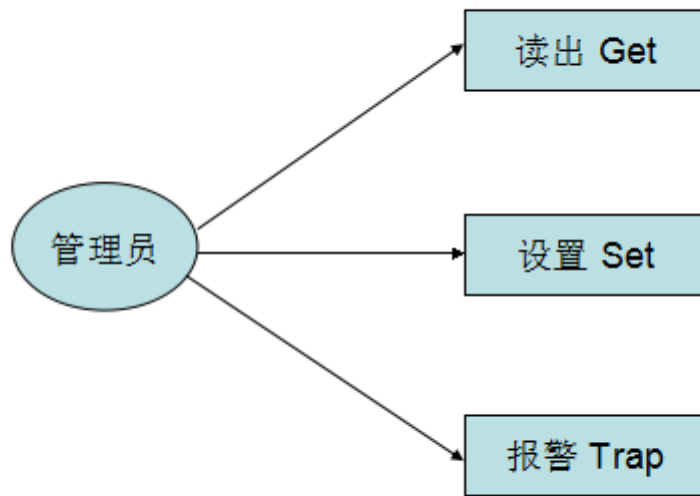
定义：



被管设备
包含若干
被管对象
它们的数据被收集在
*Management
Information
Base (MIB)*

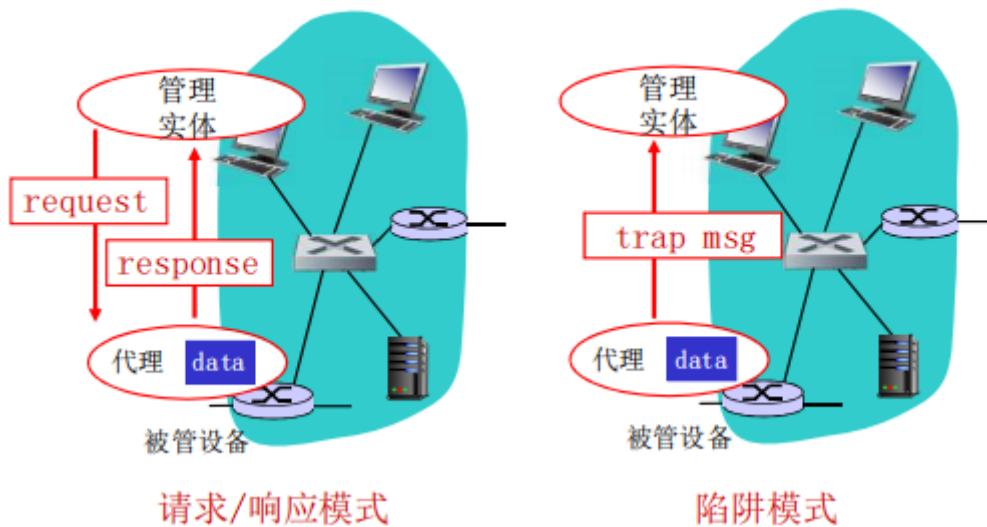
6.4 SNMP(Simple Network Management Protocol)

SNMP用于网络设备的管理。SNMP的工作方式：管理员需要向设备获取数据，所以SNMP提供了“读”操作；管理员需要向设备执行设置操作，所以SNMP提供了“写”操作；设备需要在重要状况改变的时候，向管理员通报事件的发生，所以SNMP提供了“Trap”操作。



- Get: 读取网络设备的状态信息
- Set: 远程配置设备参数
- Trap: 管理站及时获取设备的重要信息

SNMP的两种模式:



SNMP报文类型:

报文类型	功能
GetRequest GetNextRequest GetBulkRequest	管理实体-代理：“给我数据” (instance,next in list, block)
InformRequest	实体-实体：给你MIB值
SetRequest	实体-代理：set MIB value
Response	代理-实体：值，对请求的响应
Trap	代理-实体：异常事件的报告

[SNMP原理与实战](#)